

Design Document

Author: Menna Jaheen

Overview

How I intend to build a system that turns a photograph of a purchase order into a saved Order, Debtor and linked Invoice inside Fakturama 2.x, a Java/SWT Eclipse RCP invoicing application on Windows. Stack: Python; Google Gemini for extraction; pywinauto over Microsoft UI Automation for the UI; plain Win32 for windows and processes.

A design that assumes the UI is honest produces a run that reports success and leaves no invoice behind. Hence the governing principle:

Every write is confirmed by a subsequent read, and anything that cannot be confirmed stops the run.

Control-discovery and grounding

Fakturama is SWT and draws its own widgets, so a Win32 automation backend would see almost nothing — UI Automation is the only interface that can see inside it. I expect its coverage to be uneven, normal for Java toolkits on Windows, so the design must degrade gracefully rather than assume a complete tree.

Every field is described declaratively as a FieldSpec — a name, a lookup strategy, and what that strategy needs — held in one central file per screen, each entry carrying a comment naming the inspection dump that confirmed it. A later reader should never have to work out whether an entry is a verified fact or a guess.

The ladder. Each control uses the highest rung that applies, decided from a dump of the real screen rather than from what seems likely:

1. **By accessible name** — a unique, stable name.
2. **By position relative to a label** — find the label, take its parent, select the first child of the wanted type appearing *after* it in tree order. The "after" matters: one parent commonly holds several label-and-field pairs, so a first match anywhere in the parent fills the wrong box.
3. **By measured coordinate** — no accessible representation at all. I expect at least one, since custom-painted SWT tables frequently expose no rows.

A fourth rung, vision grounding (screenshot plus description), is designed in as an extension point but not built in the first pass: it puts a model call inside a UI lookup, and I'd rather learn how many controls truly need it.

Two rules held everywhere. Never key a lookup on the developer-assigned automation ID — these aren't guaranteed stable across restarts, and an unstable key gives the worst failure mode available: it works in development and breaks on the next launch. And an ambiguous match is an error, never a choice; two candidates means a defect in the spec, and taking the first would hide it. That covers dropdown options and result rows too, where "two records match" is a case a human must decide.

Writing values is a separate problem from finding controls, because I expect the obvious approach to fail silently: UIA can write a control's text directly, but for a toolkit drawing its own widgets that write may never fire the listeners that commit it, leaving a field that looks right all session and is absent after saving. So all input goes through real synthetic keystrokes — click, select-all, type, tab out — escaped first, since automation keyboard syntax assigns meaning to `+`, `%` and `^`, which appear in real data such as international phone numbers.

Connecting uses Win32 first, each step gating the next: close any leftover instance, launch, find the main window by enumerating windows owned by that process (matching ownership, not title, which varies by version and workspace), poll `IsHungAppWindow` until it stops reporting as hung, and only then hand that handle to UIA. The reason is mechanical: a UIA call is serviced by the target's own UI thread, and while an Eclipse RCP app boots that thread isn't pumping messages, so the call blocks past any timeout requested — that timeout being a client-side retry loop, not a cancellation. Every later lookup is scoped to that one window's subtree. Editor tabs are resolved once, at open time, and kept: Eclipse renames a tab after saving, which breaks any later title-based lookup.

Contingency for a control UIA cannot see: drive it as a person would — land on a row with a click and the arrow keys, copy with `Ctrl+C`, read the clipboard. That means coordinates, so three rules make them survivable. *Measure, never estimate* — a small tool records real clicks and reports offsets and the distance between them, so the row height reports itself; the result is stored as data, making a re-measure a one-line edit. *Refuse impossible clicks* — every computed point is checked against the dialog's live rectangle, so stale geometry declines to act rather than clicking through to the window beneath. *Confirm before committing* — the chosen row is read back and compared before the confirm button is pressed. I also expect the app to throw its own errors on ordinary paths, and an unhandled modal would block everything after it, so the design needs somewhere to put the target's own bugs.

Image extraction

One call to Gemini in structured-output mode, constrained by a Pydantic schema, so the model cannot return a shape the program is unprepared for and the result arrives validated. The schema mirrors what the flow needs rather than what the document contains: order date and reference; debtor identity and addresses; payment method, status and date; line items with SKU, quantity, price, discount and VAT; and the totals.

Normalisation lives in the schema, not a separate cleaning pass — ISO dates, floats for money, postal codes as strings because leading zeros are meaningful, optional fields defaulting to

empty. A second pass means a second representation of the same data, and two representations drift.

Each result is cached to disk, keyed to the source image. Debugging the automation means running the same order many times; re-extracting is slow, costs money and isn't deterministic. Pinning the data makes a failure reproducible, so a change in behaviour is a change in the automation rather than the model.

The arithmetic is validated before the UI is touched: each line's $\text{quantity} \times \text{price} \times (1 - \text{discount})$ against its line total, and the lines against the stated totals. A mismatch stops the run before Fakturama launches. A misread quantity is exactly the error automation turns into a confidently wrong invoice.

Extraction knows nothing about Fakturama — it produces an object describing an order, not instructions for filling boxes. Mapping values onto screens belongs to the step that owns each screen, which keeps extraction reusable and testable without a running application. Steps themselves raise an exception for cases needing human judgement rather than returning a status string a caller can forget to check; it surfaces once, at the top, as its own exit code.

Tradeoffs

Chosen	Rejected	Reasoning	Accepted cost
Accessibility tree first, coordinates last	Screenshot-and-click throughout	Named lookups survive moves and layout shifts; coordinates survive none	Every screen must be inspected first
Synthetic keystrokes for all input	Setting values through the API	The fast path may not persist, and fails silently when it doesn't	Slower; occupies mouse and keyboard
Verify every write by reading it back	Trust the operation that reported success	The app may accept input it does not save	Roughly doubles interactions per field
Read a whole result grid, then decide	Stop at the first match	Detecting ambiguity requires seeing every candidate	Extra passes over a grid whose first row often matches
Stop for review on ambiguity	Pick the best candidate	A wrong customer on an invoice is worse than an unfinished run	Lower automation rate

Chosen	Rejected	Reasoning	Accepted cost
Vision grounding designed in, not built	Implement it up front	Unknown how many controls need it; adds a model call inside a lookup	One control class may need a coordinate workaround meanwhile

De-risking: what I'd do first

The assumptions above are load-bearing and cheap to test, so the first session goes on an inspection spike before any flow code. Dump the control tree of the main window and one editor, to settle how much UIA can see and which rung each field needs. Restart and dump again, diffing the automation IDs — ten minutes, and it settles the rule against keying on them. Write one field, save, reopen and read it back — the pivotal experiment, deciding whether the direct value-set path is usable at all. Open the customer picker with a matching record on screen and dump it; if the rows are absent, the coordinate contingency is needed, better known on day one than half-way through.

Implementation order follows: connect and open an Order; then the Debtor branch end to end, since it exercises search, exact-match, creation, save and re-verification — every mechanism the rest reuses; then Products; then the Order save and the linked Invoice.